체크박스(☐)를 클릭하면 체크되고, Edge·크롬·Adobe Reader 에서 저장(Ctrl+S)하면 남습니다.

Git 작업 규칙

2026-10-07 · 작성 Sung Hyung Lee (PM) · 6인 팀 공용 · 저장소: sunghyunglee6334-web/group2-project

한눈에 보기

GitHub 는 PM 이 관리합니다. 팀원은 **자기 역할 브랜치에만** 올리고, main 에 합치는 것(Merge)·브랜치 만들기/지우기·설정 변경은 **PM 만** 합니다. 팀원이 지킬 것은 세 가지입니다: ① **작업 시작 전 최신 main 을 내 브랜치에 반영** ② `git add` **전에** `git status` **로** `data/` **·사진·** `Zone.Identifier` **확인** ③ **충돌이 나면 혼자 해결하지 말고 PM 에게.** 맨 뒤 **9장 따라 하기**에 그대로 치면 되는 명령을 모아 두었습니다.

규칙	내용	막아 주는 문제
1	작업 시작 전 최신 main 을 내 브랜치에 반영	commit histories 불일치, 큰 충돌
2	폴더 복사·`git init` 금지, 코드는 `kimchi_labeler/src` 안에서만	src 중복, 코드 분산
3	`git add` 전에 `git status`	데이터·사진·Zone.Identifier·오타 파일 업로드
4	충돌은 혼자 해결하지 않기	README 충돌 표시 업로드, 작업 유실
5	브랜치 이동 전 `git status`	변경 덮어쓰기 오류, 작업 유실
6	main 은 PM 만 Merge (PR 필수)	검토 안 된 코드·데이터가 main 에 들어감
7	라벨링 결과는 GitHub 로 공유하지 않기	NDA 데이터 업로드
8	내 브랜치에서는 내 담당 파일만 수정	팀원 간 충돌

1. GitHub 관리 체계 (PM 담당)

오늘 생긴 문제(데이터 업로드, src 중복, 충돌, README 깨짐)는 누구나 main·브랜치·설정을 바꿀 수 있어서 생겼습니다. 앞으로 **GitHub 관리는 PM 이 맡고**, 팀원은 정해진 범위 안에서만 작업합니다.

구분	PM 만 한다	팀원이 한다
main	PR 검토 후 Merge, main 보호 설정	읽기만 (`git pull origin main`)
브랜치	새 브랜치 만들기·이름 정하기·지우기	자기 역할 브랜치에 push
PR	Files changed 검토, Merge 또는 수정 요청	자기 브랜치 → main PR 만들기
충돌	해결 방법 결정 (담당자와 함께)	발견하면 멈추고 PM 에게 보고
설정	Settings, Collaborators, 공개 여부	건드리지 않음
정리	다 쓴 브랜치·잘못 올라간 파일 정리	발견하면 PM 에게 보고

☐ 팀원은 main 에 직접 push 하지 않는다.

☐ 팀원은 PR 을 직접 Merge 하지 않는다 (자기 PR 이라도 PM 이 Merge).
☐ 팀원은 브랜치를 새로 만들거나 지우지 않는다. 필요하면 PM 에게 요청한다.
☐ 팀원은 Settings 를 바꾸지 않는다.
☐ PM 은 main 보호 설정을 켜 둔다 (7장).

2. 브랜치 구조 (각자 브랜치)

```
main                    ← 최종본. PM 만 Merge. 모두 여기서 최신본을 받음
  ├── feature/pm          ← PM (설정 · 스크립트 03 · 07 · README)
  ├── feature/gui         ← GUI (src/ui, 화면)
  ├── feature/coord       ← COORD (src/bbox 좌표, 필요 시 PM 이 생성)
  ├── feature/storage     ← STORAGE (저장 · 불러오기, 필요 시 PM 이 생성)
  ├── feature/qa          ← QA (validation, tests, test_report)
  └── feature/manifest    ← DOC (manifest · stats · docs 문서)

chore/… docs/… fix/…      ← PM 이 잠깐 쓰는 정리용 브랜치 (Merge 후 삭제)
```

역할	내 브랜치	담당자	주로 고치는 곳
PM	`feature/pm`		`config.py`, `configs/`, 스크립트 03·07, README
GUI	`feature/gui`		`src/ui/`
COORD	`feature/coord`		`src/bbox/view.py`, `canvas_editor.py`
STORAGE	`feature/storage`		`src/bbox/session.py`, `src/yolo/`
QA	`feature/qa`		`src/validation/validator.py`, `tests/`, `reports/test_report.md`
DOC	`feature/manifest`		`src/manifest.py`, `validation/stats.py`, `docs/`

- **내 브랜치는 계속 씁니다.** PR 이 Merge 되어도 지우지 않고, 다음 날 최신 main 을 반영해서 이어서 씁니다.
- `feature/kimchi-labeler`, `feature/labeling`, 예전 `chore/…`·`docs/…` 브랜치는 PM 이 확인 후 정리합니다.
- **다른 사람 브랜치에는 push 하지 않습니다.** 남의 담당 파일을 고쳐야 하면 담당자 또는 PM 에게 요청합니다.

☐ PM: 역할별 담당자 이름을 위 표에 채우고 팀에 공지
☐ 팀원: 내 브랜치 이름을 확인 (`git branch -a`)

3. 하루 흐름 (각자 브랜치 기준)

시간	팀원	PM
시작 회의 (10:00 / 11:00)	최신 main 받기 → **내 브랜치로 이동 → main 반영** (9장 ②)	전날 Merge 내용 공지
작업 중	내 담당 파일만 수정, 수시로 commit (push 는 마감 때 해도 됨)	보고 받고 판단

17:20 마감 ①	`git status` 확인 → commit → **내 브랜치에 push** → **PR 만들기** (9장 ③④)	PR 목록 확인
17:35 마감 ②	PM 이 수정 요청하면 고쳐서 다시 push (같은 PR 에 자동 반영)	**Files changed 검토 → Merge** (9장 ⑥)
다음 날 시작	다시 ② 부터 (최신 main 을 내 브랜치에 반영)	-

오늘 작업을 내 브랜치에 올릴 때 일어나는 일

1. **commit**: 내 컴퓨터에 "저장 지점"을 만듭니다. 아직 GitHub 에는 없습니다.
2. **push**: 내 브랜치(`feature/gui` 등)를 GitHub 에 올립니다. **main 은 바뀌지 않습니다.** 다른 팀원 프로그램에도 아직 영향이 없습니다.
3. **PR**: "내 브랜치 내용을 main 에 넣어 주세요" 요청서입니다. PM 이 바뀐 파일을 확인합니다.
4. **Merge (PM)**: 이때 비로소 main 이 바뀝니다. 다음 날 모두가 `git pull origin main` 으로 받습니다.
5. 같은 브랜치에 다시 push 하면 **열려 있는 PR 에 자동으로 추가**됩니다. PR 을 새로 만들 필요가 없습니다.

4. 규칙 1~5 (작업할 때)

규칙 1. 작업 시작 전 최신 main 을 내 브랜치에 반영

```
git switch main
git pull origin main
git switch feature/내역할
git merge main
```

main 의 최신 코드와 `.gitignore` 가 내 브랜치에 들어와서, 데이터가 막히고 나중에 합칠 때 충돌이 작아집니다. 이 단계에서 `CONFLICT` 가 나오면 규칙 4.

☐ 매일 작업 전에 위 네 줄을 한다.

규칙 2. 폴더 복사·git init 하지 않기

저장소는 처음 한 번만 `git clone` 으로 받습니다. 폴더를 복사해서 새로 올리거나 `git init` 을 하면 Git 이 남남으로 보고 코드가 두 군데로 나뉩니다. 코드는 **`kimchi_labeler/src/` 안에서만** 고칩니다.

☐ 내 컴퓨터의 저장소는 `git clone` 으로 받은 하나뿐이다.

규칙 3. git add 전에 git status

목록에 보이면 멈추고 PM 에게	이유
`data/` 폴더	라벨·사진·작업 데이터 (NDA)
`.jpg` · `.jpeg` · `.txt` (requirements.txt 제외)	이미지·라벨 파일
`:Zone.Identifier`	Windows 꼬리표 파일
`settings.json` , `.venv`	개인 설정·가상환경
처음 보는 이상한 이름, 내 담당이 아닌 파일	오타 파일이거나 남의 파일

commit 직전 확인 명령입니다. **아무것도 안 나와야** 합니다.

```
git diff --cached --name-only | grep -E '/data/|\.jpe?g$|Zone\.Identifier|settings\.json'
```

☐ `git add` 전 `git status`, commit 전 확인 명령을 한다.

규칙 4. 충돌은 혼자 해결하지 않기

`CONFLICT` 가 나오면 **멈추고 PM 에게**. 되돌리려면 `git merge --abort`. 해결한 뒤에는 아래 결과가 **비어 있을 때만** commit 합니다.

```
grep -rn '^<<<<<<<\|^=======\|^>>>>>>>' --include='*.py' --include='*.md' .
```

☐ `CONFLICT` 는 PM 과 함께 해결한다.

규칙 5. 브랜치 이동 전 git status

`working tree clean` 이 아니면 먼저 commit 하거나 `git stash` 로 보관한 뒤 이동합니다. 파일을 고쳤으면 VS Code 에서 **Ctrl+S 저장**했는지 확인합니다.

```
git stash           # 커밋 안 한 변경 잠시 보관
git switch 다른브랜치
git stash pop       # 돌아와서 꺼내기
```

☐ 브랜치 이동 전 `git status` 가 clean 인지 본다.

5. 규칙 6~8 (올릴 때)

규칙 6. main 은 PM 만 Merge

- 항상 **내 브랜치에 push → PR → PM 이 검토 후 Merge**.
- 커밋 메시지는 `feat:` 새 기능, `fix:` 버그 수정, `docs:` 문서, `chore:` 정리 로 시작합니다.
- PR 제목: `[역할] 날짜 한 줄 요약` 예: `[GUI] 10/08 목록 스크롤바 수정`

규칙 7. 라벨링 결과는 GitHub 로 공유하지 않기

- 라벨링 결과(`data/work`)는 **공유폴더·USB 로만** 모으고 DOC 가 `04_merge.py` 로 합칩니다.
- 라벨 파일을 "올려야 하나?" 싶으면 무조건 올리지 않습니다. 오늘 문제 1번의 원인입니다.

규칙 8. 내 브랜치에서는 내 담당 파일만

- 2장 표의 "주로 고치는 곳" 밖의 파일은 고치지 않습니다. 같은 파일을 두 사람이 고치면 충돌이 납니다.
- 남의 파일을 고쳐야 하면 담당자에게 요청하거나 PM 과 정합니다.

☐ PR 은 내 브랜치 → main 으로만 만든다.

☐ 라벨링 결과는 공유폴더로만 전달한다.

☐ 내 담당 파일만 수정한다.

6. 금지 명령

명령	이유
`git init`	저장소를 새로 만들어 기록이 따로 놀게 됨

`git push origin main`	검토 없이 main 이 바뀜 (PM 만 PR 로 Merge)
`git push --force` / `-f`	다른 사람 작업을 덮어써 지울 수 있음
`git push origin --delete …`	브랜치 삭제는 PM 만
`git add -f data/…`	제외 규칙을 무시하고 데이터를 올림
`git add -A` (git status 확인 없이)	쓰레기·데이터 파일이 같이 올라감
다른 사람 브랜치에 push	남의 작업과 섞임

7. PM 관리 업무

main 보호 설정 (처음 한 번)

1. 저장소 **Settings → Branches → Add branch protection rule** (화면에 **Rulesets** 로 보이면 **Rules → Rulesets → New branch ruleset**, 대상 `main`)
2. Branch name pattern: `main`
3. **Require a pull request before merging** 체크 (승인 수는 0 으로 두어도 됨)
4. **Do not allow force pushes**, **Do not allow deletions** 확인
5. **Create / Save changes**

이렇게 하면 누구도 main 에 직접 push 할 수 없고, PR 을 거쳐야만 main 이 바뀝니다.

PR 검토 체크 (매일 17:35)

☐ PR 이 `내 브랜치 → main` 방향인가

☐ Files changed 에 `data/`, `.jpg`, `.txt`(라벨), `Zone.Identifier`, `settings.json` 이 없는가

☐ `<<<<<<<` 충돌 표시가 없는가

☐ 담당 범위 밖 파일이 바뀌지 않았는가

☐ Merge 후 PM 컴퓨터에서 `git pull origin main` → `bash run.sh` 실행 확인

☐ 서기에게 오늘 Merge 한 PR 목록 전달

주기 정리

- 다 쓴 임시 브랜치(`chore/…`, `docs/…`, `fix/…`)는 Merge 후 삭제합니다. 역할 브랜치는 지우지 않습니다.
- 아래 명령으로 GitHub 전체 브랜치에 데이터가 없는지 가끔 확인합니다. 모두 **0** 이어야 합니다.

```
git fetch origin --prune
for b in $(git branch -r | grep -v HEAD); do echo "$b : $(git ls-tree -r --name-only $b | grep -c
'/data/\|\.jpg$\|\.jpeg$')"; done
```

8. 문제가 생겼을 때

화면에 나온 것	할 일
`CONFLICT`	멈추고 PM 에게. 되돌리려면 `git merge --abort`
`Your local changes … would be overwritten`	`git stash` → 이동 → `git stash pop`
`nothing to commit` (고쳤는데)	VS Code 에서 Ctrl+S 저장 후 다시 `git status`
`entirely different commit histories`	main 과 상관없는 브랜치 → PM 에게
`refusing to merge unrelated histories`	내 브랜치가 main 과 따로 만들어진 것 → PM 이 main 에서 브랜치를 새로 만들어 줌 (내 수정 파일은 PM 과 함께 옮김)
`protected branch` / push 거절	main 에 push 하려 한 것 → 내 브랜치로 push
`rejected … fetch first`	GitHub 의 내 브랜치가 더 최신 → `git pull origin feature/내역할` 후 다시 push
nano 편집 화면	`Ctrl+O` → `Enter` → `Ctrl+X`
git status 에 `data/` ·사진	절대 add 하지 말고 PM 에게

9. 따라하기 (이 순서대로 치면 됩니다)

앞의 내용을 실제 순서대로 모았습니다. 팀원은 ①~⑤, PM 은 ⑥~⑦ 입니다. 명령은 WSL 터미널에 한 줄씩 입력합니다. `feature/내역할` 은 2장 표의 내 브랜치 이름으로 바꿔 씁니다 (예: `feature/gui`).

① 처음 한 번만: 저장소 받기와 설치 (팀원)

```
git clone https://github.com/sunghyunglee6334-web/group2-project.git ~/group2_project
cd ~/group2_project/kimchi_labeler
bash setup_wsl.sh
git config --global user.name "내 이름"
git config --global user.email "내 GitHub 이메일"
cd ~/group2_project
git switch feature/내역할           # PM 이 만들어 둔 내 브랜치로 이동
```

☐ "모든 테스트 통과" · "설치 완료." 확인, 실제 데이터는 `kimchi_labeler/data/raw` 에 (Git 에는 안 올라감)

② 매일 작업 시작 (팀원, 시작 회의 직후)

```
cd ~/group2_project
git status                  # working tree clean 인지
git switch main
git pull origin main        # 최신 main 받기
git switch feature/내역할     # 내 브랜치로
git merge main              # 최신 main 을 내 브랜치에 반영 (CONFLICT 나오면 PM)
```

☐ 내 브랜치에 있고(`git branch` 에 `*` 표시), main 을 반영했다.

③ 작업 후 내 브랜치에 올리기 (팀원, 17:20)

```
git status                                    # 바뀐 파일 확인 (data/ · 사진 · Zone · 남의 파일 없는지)
git add kimchi_labeler/src/ui/app.py          # 올릴 파일만 지정 (여러 개면 띄어서)
git diff --cached --name-only | grep -E '/data/|\.jpe?g$|Zone\.Identifier|settings\.json'
                                              # 아무것도 안 나와야 함
git commit -m "fix: 저장 후 목록 갱신 오류 수정"
git push origin feature/내역할                 # main 이 아니라 내 브랜치
```

☐ 확인 명령 결과가 비어 있었다.

☐ `feature/내역할` 에 push 했다.

④ GitHub 에서 PR 만들기 (팀원)

1. 저장소 페이지 위 노란 줄 **Compare & pull request** 클릭
   (안 보이면 **Pull requests → New pull request** → `base: main` ← `compare: feature/내역할`)
2. 제목 `[역할] 날짜 한 줄 요약`, 설명에 무엇을 왜 바꿨는지 한두 줄
3. **Create pull request** → PM 에게 "PR 올렸습니다" 알림. **Merge 버튼은 누르지 않습니다.**
4. 이미 열린 PR 이 있으면 새로 만들지 않습니다. push 만 하면 자동으로 추가됩니다.

⑤ PM 이 Merge 한 뒤 / 다음 날 (팀원)

내 브랜치는 지우지 않고, 다음 날 ② **부터** 다시 합니다. `git merge main` 으로 다른 팀원 작업까지 내 브랜치에 들어옵니다.

⑥ PR 검토와 Merge (PM, 17:35)

1. **Pull requests** → PR 열기 → **Files changed** 에서 7장 체크 항목 확인
2. 문제 있으면 **Review changes → Request changes** 로 수정 요청 (팀원이 고쳐 다시 push)
3. 문제 없으면 **Merge pull request → Confirm merge**. 역할 브랜치는 **Delete branch 누르지 않음**
4. PM 컴퓨터에서 확인:

```
cd ~/group2_project
git switch main
git pull origin main
cd kimchi_labeler && bash run.sh     # 프로그램 정상 실행 확인
```

⑦ 처음 한 번 (PM)

☐ main 보호 설정 켜기 (7장)

☐ 역할 브랜치 만들기 (없는 것만): `git switch main` → `git pull origin main` → `git switch -c feature/coord` → `git push -u origin feature/coord`

☐ 기존 역할 브랜치(`feature/qa`, `feature/pm`, `feature/manifest` 등)가 main 과 기록이 이어져 있는지 확인: `git merge-base origin/main origin/feature/qa` 결과가 비어 있으면 따로 만들어진 브랜치 → 담당자와 수정 파일을 확인한 뒤 main 에서 새로 만들기

☐ 2장 표에 담당자 이름 채워 팀 공지, 안 쓰는 옛 브랜치 정리